UNIVERSIDAD TECNOLÓGICA DE PEREIRA

INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

HPC: HIGH PERFORMANCE COMPUTING

Análisis del Algoritmo de Jacobi Iterativo para la Ecuación de Poisson 1D

Autor: Jorge Luis López Grajales

Asignatura: HPC: High Performance Computing

Docente: Ramiro Andrés Barrios Valencia

Fecha: 26 de marzo de 2026

Pereira – Colombia

1. Análisis del Algoritmo de Jacobi Iterativo

El problema consiste en resolver la ecuación de Poisson en una dimensión:

$$-u'' = f(x)$$

en un dominio $[a, b]$ con condiciones de frontera de Dirichlet.

1.1. Formulación matemática y discretización

Para aproximar la solución, dividimos el dominio en N puntos con un espaciado constante h. Aplicando diferencias finitas centradas de segundo orden, la ecuación diferencial se transforma en el sistema:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f(x_i).$$

Esto genera un sistema lineal $Au = b$, donde A es una matriz tridiagonal.

1.2. Lógica de la iteración de Jacobi

El método de Jacobi descompone la matriz A en D (diagonal) y $(L + U)$ (triangulares). La actualización de cada nodo para la iteración $k + 1$ es:

$$u_i^{(k+1)} = \frac{1}{2}\left(u_{i-1}^{(k)} + u_{i+1}^{(k)} + h^2 f_i\right).$$

Nota clave: La independencia de los cálculos en cada nodo i (solo dependen de valores de la iteración previa k) convierte a Jacobi en un algoritmo naturalmente paralelo (*embarrassingly parallel*).

2. Opciones de concurrencia y optimización

El análisis de implementaciones paralelas y optimizaciones de sistema revela diferencias críticas en eficiencia.

2.1. Threads (hilos) vs. Fork (procesos)

- **Threads:** Utilizan un espacio de memoria compartido. Son ligeros en creación y permiten comunicación inmediata para acceder a los vectores de la malla.
- **Fork:** Crea procesos independientes con su propio espacio de direccionamiento. Aunque existe *copy-on-write*, la sincronización de fronteras de subdominios en cada iteración genera latencia significativa.

2.2. Optimización de memoria

- **Gestión del mallado:** Jacobi requiere dos vectores (u_{old} y u_{new}).
- En hilos, ambos residen en memoria global accesible por todos.
- En procesos, se requiere memoria compartida explícita (por ejemplo, `mmap` o `shmget`), lo que añade complejidad y sobrecosto.

2.3. Optimización de CPU

- **Compilador:** Banderas como `-Ofast` habilitan vectorización (SIMD) y desenrollado de bucles.
- En problemas pequeños ($K < 11$), la versión secuencial optimizada puede ser más eficiente que la paralela por ausencia de sincronización.
- **Complejidad:** El costo total crece como $O(n_k^3)$.
- Las optimizaciones deben equilibrar reducción del tiempo por iteración (paralelismo) y reducción del número de iteraciones (Gauss-Seidel, SOR).

3. Análisis de resultados de desempeño

Basado en las gráficas de *speedup* y tiempo obtenidas en los barridos de K, se observan las siguientes causas:

A. Sobrecosto de gestión (*overhead*)

Para valores bajos de malla ($K \leq 9$), el *speedup* es menor que 1.
Causa: El tiempo del kernel del SO en crear contexto de hilos o clonar procesos es mayor que el tiempo de cómputo de la malla. En este rango, la versión secuencial optimizada domina.

B. Relación cómputo/comunicación

El *speedup* de hilos supera 2.0 a partir de $K = 12$.
Causa: Al crecer la malla, la carga computacional por iteración aumenta y la penalización de la barrera de sincronización se vuelve relativamente pequeña.

C. Eficiencia de la arquitectura de memoria

Los hilos superan consistentemente a los procesos en escalabilidad.
Causa: El aislamiento de memoria de *fork* penaliza el rendimiento. Los procesos introducen más contención en planificador y bus de datos, reflejada en picos de inestabilidad temporal.

D. Impacto de la tolerancia

En barridos de tolerancia, el *speedup* se mantiene casi constante al aumentar la precisión.
Causa: Una tolerancia más estricta (por ejemplo, 10^{-6}) incrementa iteraciones, pero no la carga por iteración. La proporción cómputo/sincronización se mantiene.

3.1. Gráficas de desempeño

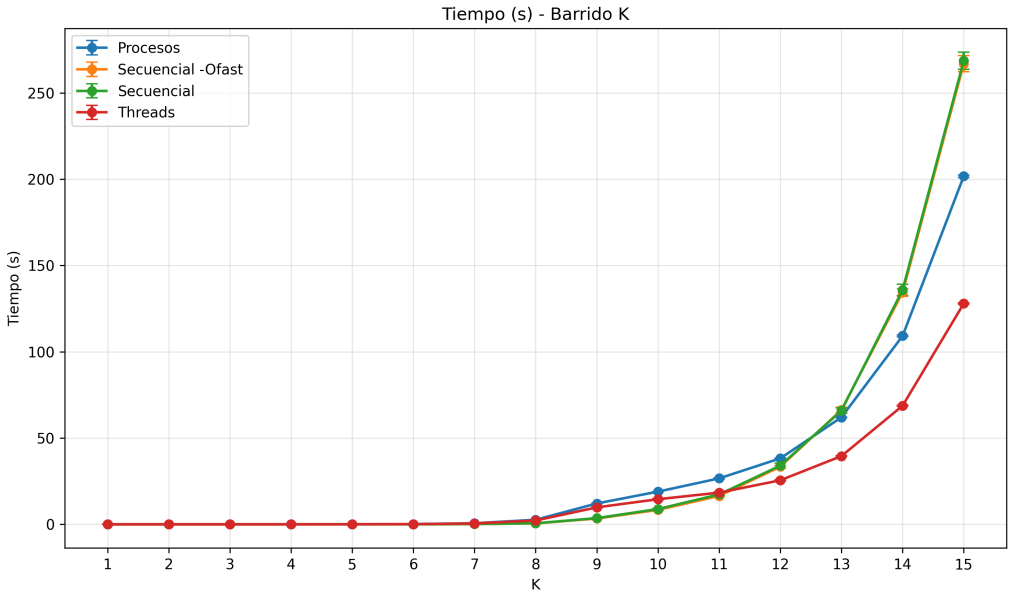

Figura 1: Comparación de tiempos para barrido en K.

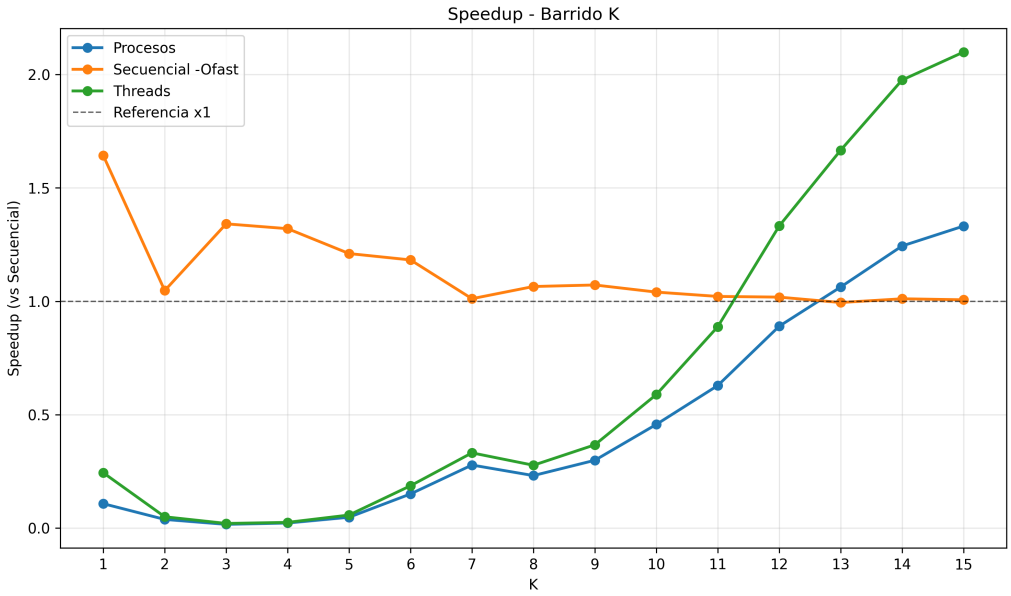

Figura 2: Speedup (base secuencial) para barrido en K.

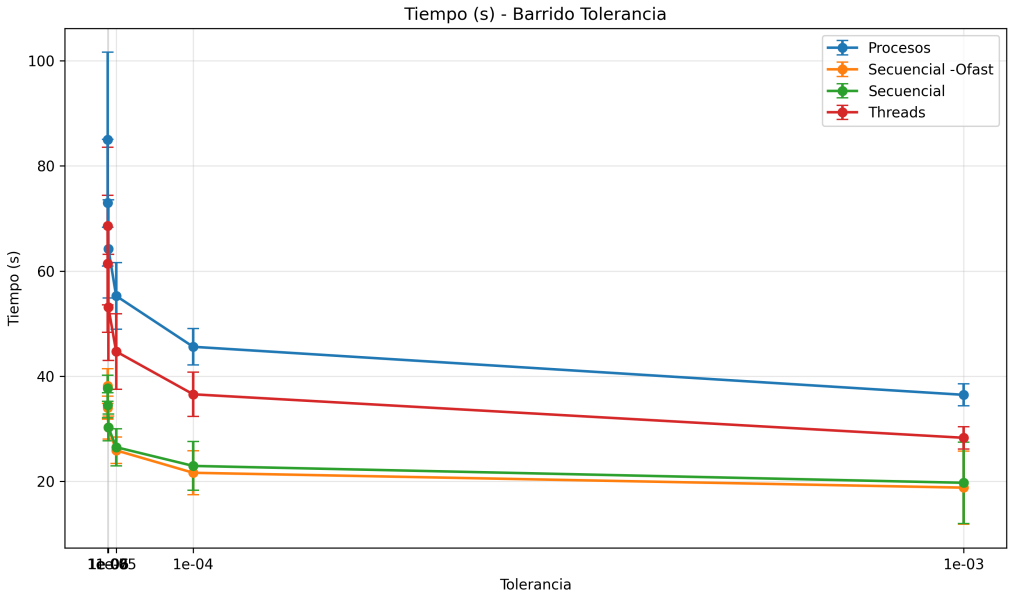

Figura 3: Comparación de tiempos para barrido de tolerancia.

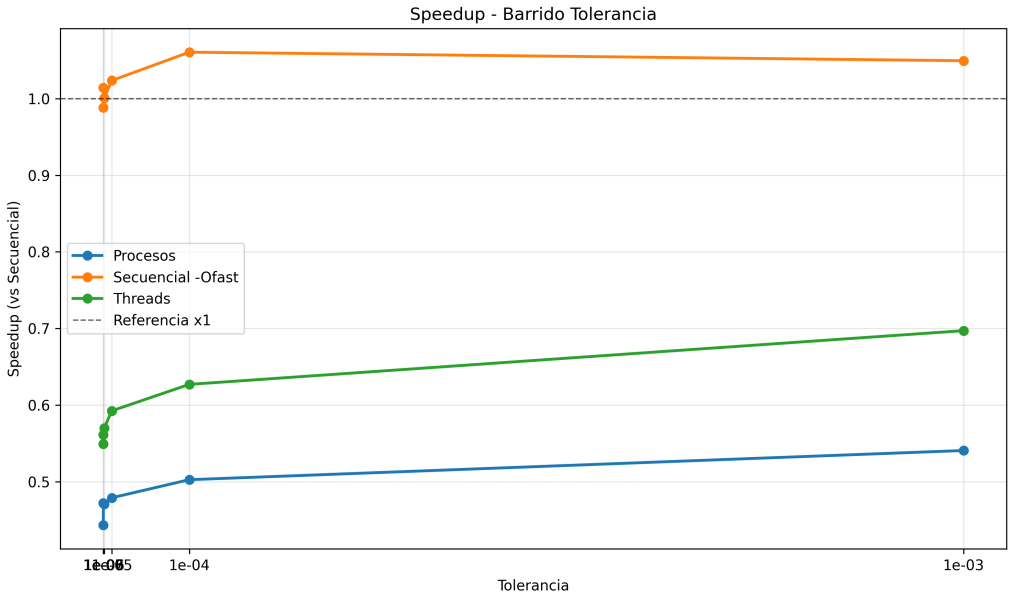

Figura 4: Speedup (base secuencial) para barrido de tolerancia.

Resumen de hallazgos

- Los hilos son la opción preferida para Jacobi por su baja latencia en memoria compartida.
- Conviene paralelizar solo en mallas grandes: para $K < 10$, el paralelismo puede ser contraproducente.
- El cuello de botella principal en Jacobi es la barrera de sincronización obligatoria, que limita el *speedup* máximo.